

Logic Programming for Inference on Quantum Computer

Davide Camino



UNIVERSITÀ
DI TORINO

`davide.camino@edu.unito.it`

Obiettivo del lavoro

Reasoning su quantum computer

Base di conoscenza classica + query

Embedding su quantum computer

Esecuzione e recupero risultati

Due paradigmi di programmazione

Quantum Gate

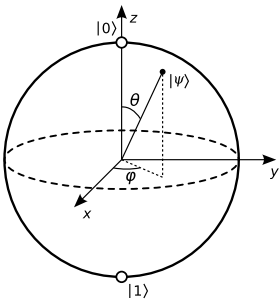


Figure: A single *qubit* gate can be seen as a rotation on the Bloch Sphere

Quantum Annealing

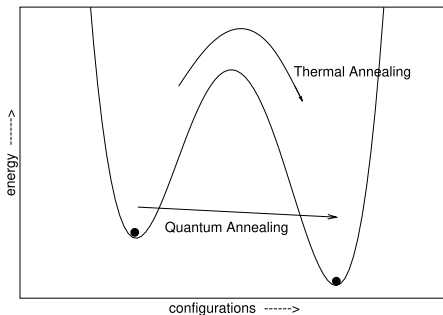


Figure: Tunneling effect (K. Chakrabarti, A. Das)

QA-Prolog

Il progetto

- Sviluppato da Scott Pakin
- Python + GO
- Proof of concept
- Trasformazioni successive
- Prolog + Query $\rightarrow \mathcal{H}_f$
- Interagisce direttamente con il solver
- Raccoglie e organizza i risultati

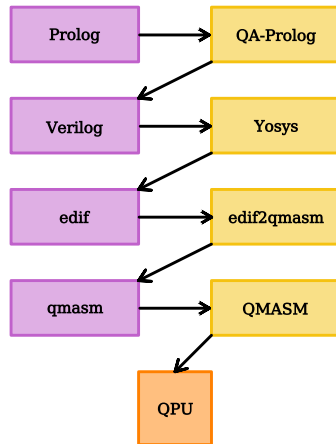


Figure: Transformation pipeline

QMASM

$\mathcal{H}_f$ simbolica $\rightarrow \mathcal{H}_f$ fisica

Cos'è

- Quantum macro assembler
- Si interfaccia con Ocean

Cosa permette di fare

- Riferimento simbolico a *qubit*
- *Qubit* “pinnati” a TRUE o FALSE
- Incapsulare pattern in macro
- Creazione di librerie di macro
- Organizzazione output

```
# Y = A OR B
!begin_macro OR
$A 0.5
$B 0.5
$Y -1
```

```
$A $B 0.5
$A $Y -1
$B $Y -1
!end_macro OR
```

Figure: or gate

Creiamo il file `gates.qasm`

QA-Prolog

- Traduzione da Prolog a Verilog
- Wrapper per tutta la Pipeline
- Risultati formato Human Readable

QA-Prolog

- Traduzione da Prolog a Verilog
- Wrapper per tutta la Pipeline
- Risultati formato Human Readable

$$A \wedge \neg(B \vee C)$$

QA-Prolog

- Traduzione da Prolog a Verilog
- Wrapper per tutta la Pipeline
- Risultati formato Human Readable

$$A \wedge \neg(B \vee C)$$

```
sat(A, B, C, Y) :-  
  or(B, C, X),  
  not(X, Z),  
  and(A, Z, Y).
```

Figure: Prolog Code

QA-Prolog

- Traduzione da Prolog a Verilog
- Wrapper per tutta la Pipeline
- Risultati formato Human Readable

$$A \wedge \neg(B \vee C)$$

```
sat(A, B, C, Y) :-  
  or(B, C, X),  
  not(X, Z),  
  and(A, Z, Y).
```

Figure: Prolog Code

```
module sat (A, B, C, Y);  
  input A, B, C;  
  output Y;  
  
  assign Y = A & ~(B | C);  
endmodule
```

Figure: Verilog Code

QA-Prolog

- Traduzione da Prolog a Verilog
- Wrapper per tutta la Pipeline
- Risultati formato Human Readable

$$A \wedge \neg(B \vee C)$$

```
sat(A, B, C, Y) :-  
  or(B, C, X),  
  not(X, Z),  
  and(A, Z, Y).
```

Figure: Prolog Code

```
module sat (A, B, C, Y);  
  input A, B, C;  
  output Y;  
  
  assign Y = A & ~(B | C);  
endmodule
```

Figure: Verilog Code

Interrogazione

```
QA-Prolog --qasm-args="--solver=tabu --postproc=opt"  
  --query='sat(A, B, C, true).' circ_sat.pl
```

```
A = true  
B = false  
C = false
```

Figure: Query Result

QA-Prolog

Esempio

Base di conoscenza

Il nemico del mio nemico è mio amico

```
hates(alice, bob).  
hates(bob, charlie).  
  
enemies(P, Q) :- hates(P, Q).  
enemies(P, Q) :- hates(Q, P).  
  
friends(A, B) :-  
    enemies(A, X),  
    enemies(X, B),  
    A \= B.
```

Figure: Enemy of my Enemy

QA-Prolog

Esempio

Base di conoscenza

Il nemico del mio nemico è mio amico

```
hates(alice, bob).  
hates(bob, charlie).  
  
enemies(P, Q) :- hates(P, Q).  
enemies(P, Q) :- hates(Q, P).  
  
friends(A, B) :-  
    enemies(A, X),  
    enemies(X, B),  
    A \= B.
```

Figure: Enemy of my Enemy

Esecuzione

```
QA-Prolog --qasm-args="--postproc=opt"  
--query='friends(P1, P2).' friends.pl
```

```
P1 = alice  
P2 = charlie  
  
P1 = charlie  
P2 = alice
```

Figure: Query Solution

Ontologie

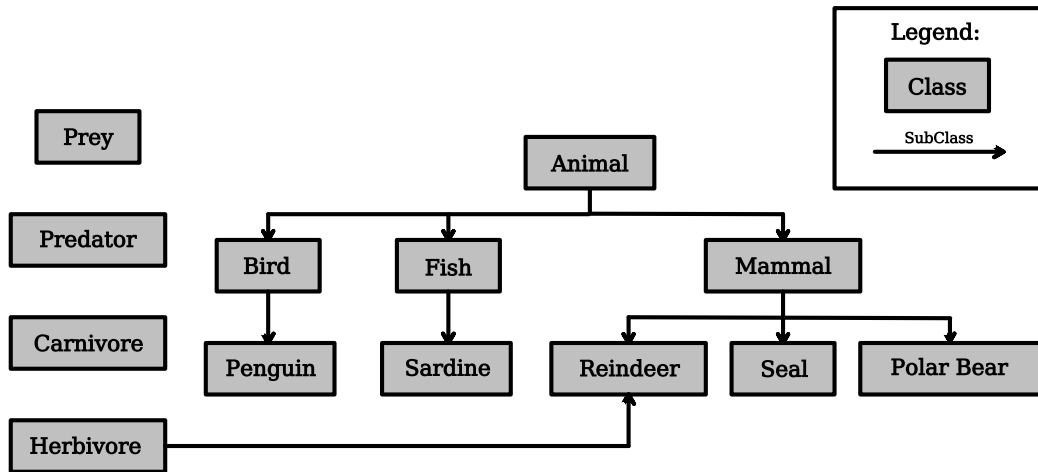


Figure: Ontology T-Box

Quantum Ontology

Traduzione in Prolog

```
class(animal).  
class(mammal).  
  
subclass(mammal, animal).  
subclass(seal, mammal).
```

Figure: Classes and subclasses

```
subClass(C1, C2) :-  
    subclass(C1, C2).  
  
subClass(C1, C2) :-  
    subclass(C1, C3),  
    subClass(C3, C2).
```

Figure: Subclasses recursive definition

Quantum Ontology

Traduzione in Prolog

```
class(animal).
class(mammal).

subclass(mammal, animal).
subclass(seal, mammal).
```

Figure: Classes and subclasses

```
subClass(C1, C2) :-
    subclass(C1, C2).

subClass(C1, C2) :-
    subclass(C1, C3),
    subClass(C3, C2).
```

Figure: Subclasses recursive definition

Unfolding

```
subClass0(C0, C1) :-
    subclass(C0, C1).

subClass1(C0, C1) :-
    subClass0(C0, C2),
    subclass(C2, C1).

subClass2(C0, C1) :-
    subClass1(C0, C2),
    subclass(C2, C1).

subClass3(C0, C1) :-
    subClass2(C0, C2),
    subclass(C2, C1).
```

Figure: Subclasses rule unfolded

```
rule_0 = 'subClass0(C0,C1) :- \n\tsubclass(C0, C1).\n'
for i in range(1, n_unfold):
    rule = 'subClass'+ str(i) + '(C0,C2) :-\n' + \
          '\tsubClass' + str(i-1) + '(C0,C3),\n' + \
          '\tsubclass(C3, C2).\n'
```

Figure: Unfolding script

Dimensione del problema

Query

- ① `?- class(animal).`
- ② `?- class(X).`
- ③ `?- subClass(seal, X).`
- ④ `?- isA(sardy, X).`
- ⑤ `?- error(X, Y, Z).`
- ⑥ `?- hasProperty(I1, ancestor, reiny_c).`

Figure: Example Query

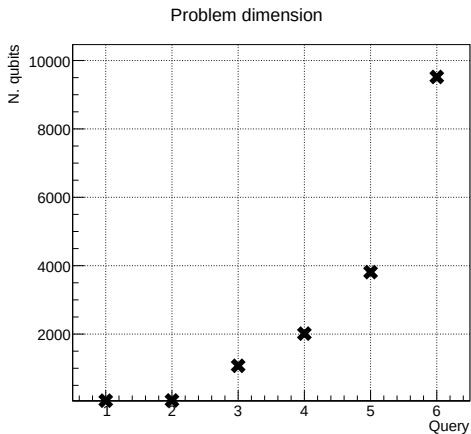


Figure: Problem Dimension

Dimensione del problema

Impatto dell'unfolding

Knowledge Base

```
class(animal).  
class(mammal).
```

```
subclass(mammal, animal).
```

Figure: Facts

```
?- subclass(mammal, animal)
```

Figure: Query

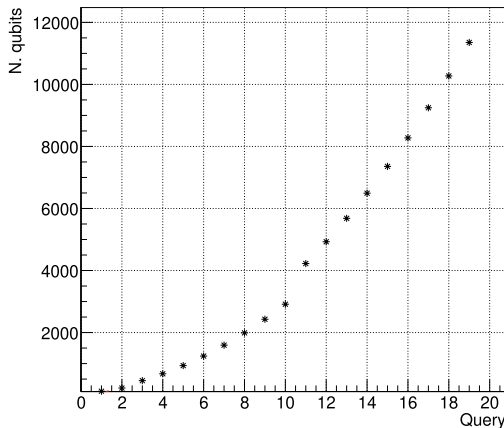


Figure: Problem Dimension

Dimensione del problema

Impatto dell'unfolding

Knowledge Base

```
class(animal).  
class(mammal).
```

```
subclass(mammal, animal).
```

Figure: Facts

```
?- subclass(mammal, animal)
```

Figure: Query

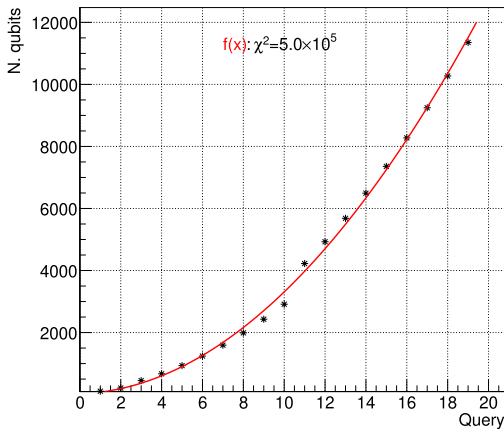
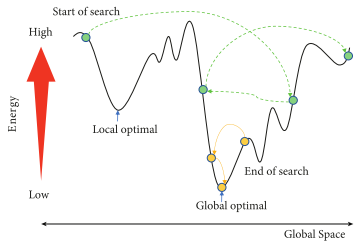


Figure: Problem Dimension

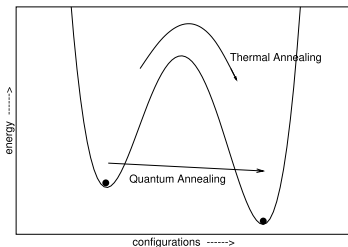
Non un solo paradigma

Problema QUBO/Ising

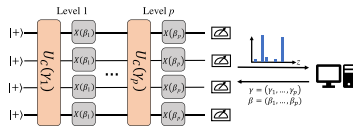
$$\mathcal{H}_f$$



(a) Simulated annealing (Li Yang)



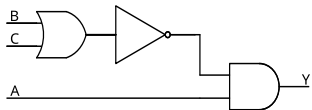
(b) Quantum annealing (K. Chakrabarti, A. Das)



(c) QAOA (Z. Zhou, Y. Du, X. Tian, D. Tao)

Figure: Single representation 3 different approaches

Esempio



```

!begin_macro sat
!use_macro AND $id000006
!use_macro NOT $id000004
!use_macro OR $id000005
$id000004.A = $id000005.Y
$id000005.A = b
$id000005.B = c
$id000006.A = a
$id000006.B = $id000004.Y
$id000006.Y = y
!end_macro sat

```

Figure: CircSat

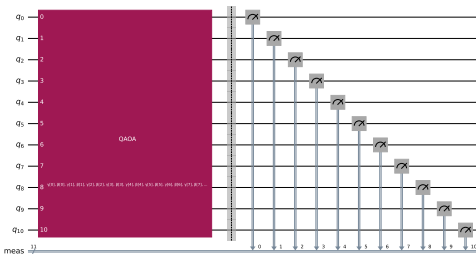
$$\begin{pmatrix}
 0.5 & 0.5 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -2 \\
 0 & 0.5 & -1 & 0 & -2 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 1 & 0 & 0 & -2 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & -0.5 & 0.5 & -1 & -2 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & -0.5 & -1 & 0 & -2 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0
 \end{pmatrix}$$


Figure: $\mathcal{H}_f$ and QAOA Quantum Circuit

Esempio

Risultato

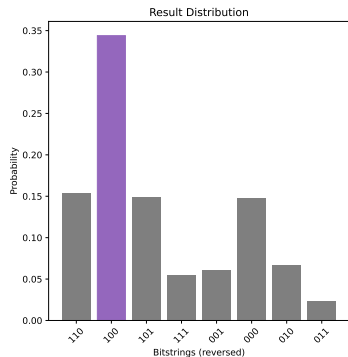


Figure: 3 “interesting” bit

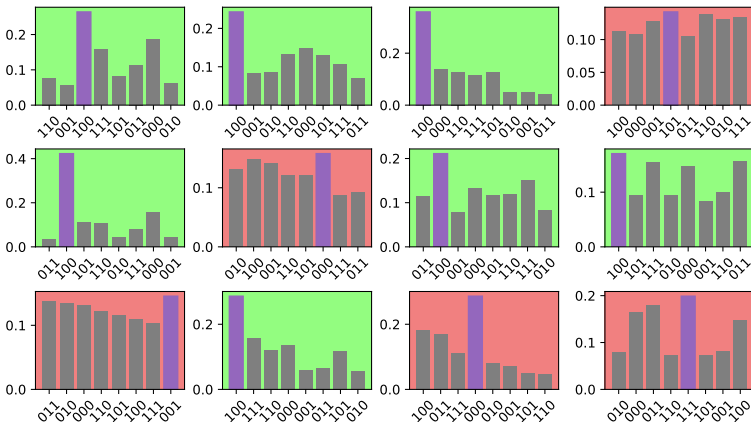


Figure: Multipe experiment runs

Lavoro Futuro

Monte

- Compilatore Ontologie → KB Prolog

Valle

- Interfacciare QMASM con altri solver

- Mantenere aggiornati i componenti
- Testare su hardware quantistico
- Confronto QA e QAOA

Lavoro Futuro

Monte

- Compilatore Ontologie → KB Prolog

Valle

- Interfacciare QMASM con altri solver

- Mantenere aggiornati i componenti (metà del lavoro di tesi)
- Testare su hardware quantistico
- Confronto QA e QAOA

Grazie per l'attenzione

Domande?



Davide Camino

davide.camino@edu.unito.it

Perché il quantum computing

Grazie a

- Superposition
- Entanglement

Violiamo

Strong Church-Turing
Thesis

$$\mathcal{H}(t) |\psi(t)\rangle = i\hbar \frac{\partial}{\partial t} |\psi(t)\rangle$$

$$\langle A_0 \otimes B_0 \rangle + \langle A_0 \otimes B_1 \rangle + \langle A_1 \otimes B_0 \rangle - \langle A_1 \otimes B_1 \rangle = 2\sqrt{2}$$

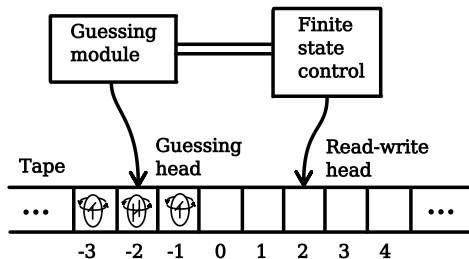


Figure: Probabilistic Turing Machine

Quantum Gate

- Formalismo molto studiato
- Gate Reversibili
- Set di porte universali
- Algoritmi di Shore, ecc.

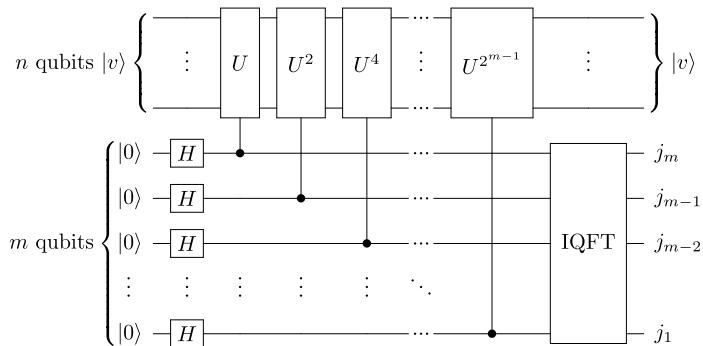
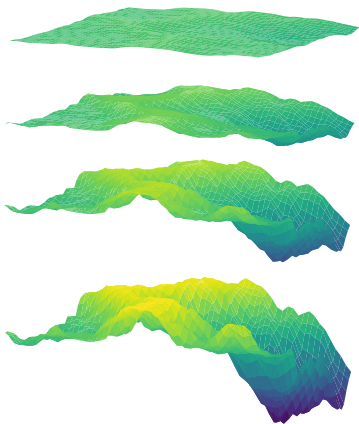


Figure: Step for Shor algorithm (Thomas G. Wong)

Quantum Annealer



- Problemi in forma QUBO
- Ispirato a Simulated Annealing
- Effetto tunneling
- Adiabatic Quantum Computing (AQC)
- $\mathcal{H}(t) = s(t)\mathcal{H}_i + (1 - s(t))\mathcal{H}_f$

Figure: Complete sovrapposizione $\rightarrow$ ground state $\mathcal{H}_f$

QMASM

Esempio: Macro

```
# Y = A OR B
!begin_macro OR
  $A 0.5
  $B 0.5
  $Y -1

  $A $B 0.5
  $A $Y -1
  $B $Y -1
!end_macro OR
```

Figure: or gate

```
# Y = NOT A
!begin_macro NOT
  $A $Y 1.0
!end_macro NOT
```

Figure: not gate

```
# Y = A AND B
!begin_macro AND
  $A -0.5
  $B -0.5
  $Y 1

  $A $B 0.5
  $A $Y -1
  $B $Y -1
!end_macro AND
```

Figure: and gate

possiamo racchiudere queste macro nel file `gates.qasm`

QMASM

Esempio: $y = x_1 \wedge \neg(x_2 \vee x_3)$

```
!include <gates>

!use_macro OR x2_or_x3
  x2_or_x3.$A = x2
  x2_or_x3.$B = x3
  x2_or_x3.$Y = $x4

!use_macro NOT not_x4
  not_x4.$A = $x4
  not_x4.$Y = $x5

!use_macro AND x1_and_x5
  x1_and_x5.$A = x1
  x1_and_x5.$B = $x5
  x1_and_x5.$Y = y
```

Figure: CircSat problem

“Pinniamo” il valore di y per ottenere l’assegnamento delle x_i che verificano la formula logica:

```
qasm --run --pin="y := true" circsat.qasm
```

Solution #1 (energy = -20.0000, tally = 647):

Variable	Value
-----	-----
x1	True
x2	False
x3	False
y	True

Figure: CircSat solution

Yosys - edif2qasm

Esempio: moltiplicazione tra interi

```
module mult (multiplicand, multiplier, product);
  input [1:0] multiplicand;
  input [1:0] multiplier;
  output [2:0] product;

  assign product = multiplicand * multiplier;
endmodule
```

Figure: Factorization problem

Traduzione in EDIF

```
yosys myfile.v synth.ys -b edif -o myfile.edif
```

Traduzione in QMASM

```
edif2qasm -o="myfile.qasm" myfile.edif
```

Esecuzione

```
qasm --run --pin="mult.product[2:0] := 110"
--solver="sim_anneal" mult.qasm
```

Solution #1 (energy = -57.5000, tally = 68):

Variable	Value
-----	-----
mult.multiplicand[0]	False
mult.multiplicand[1]	True
mult.multiplier[0]	True
mult.multiplier[1]	True
mult.product[0]	False
mult.product[1]	True
mult.product[2]	True

Figure: Factorization Solution

Update al progetto

Incompatibilità tra output edif2qasm e input QMASM

```
// Define hates(atom, atom).
module \hates/2 (A, B, Valid);
    ...
endmodule
```

Figure: Verilog

```
# hates/2
!begin_macro id00011
    ...
!end_macro id00011

# enemies/2
!begin_macro id00010
!use_macro OR $id00031
!use_macro hates/2
...

```

Figure: qasm

```
with open(file, 'r') as input:
    first = input.readline()
    second = input.readline()
    while(second_row != ""):
        if first.startswith("#") and second.startswith("!begin_macro"):
            self.name[first[2:-1]] = second[len("!begin_macro "):-1]
            first_row = second_row
            second_row = input.readline()

with open(file, 'r') as input:
    doc = input.read()
    lines = doc.splitlines()
    for line in lines:
        for word in line.split():
            if word in self.name.keys():
                line = line.replace(word, self.name[word])
            self.new_lines.append(line)
```

Figure: Preprocessing

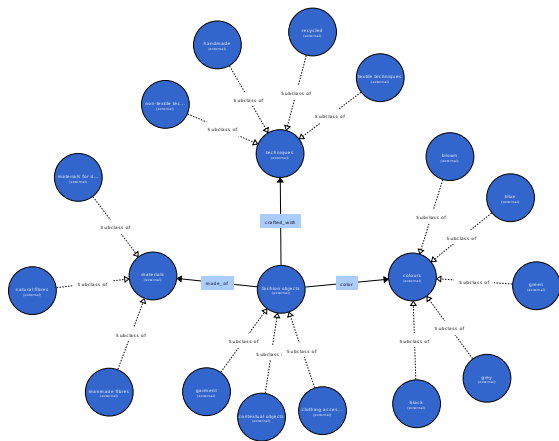
Ontologie

Cos'è un'ontologia

- Insieme di fatti
- Riguardanti un dominio di interesse
- Prevengono interpretazioni sbagliate
- Assicurano la cooperazione tra software

Come si definisce un'ontologia

- Ontology Web Language (OWL)
- Diversi “flavours”
 - OWL Full
 - OWL DL
- Classi Individui e Relazioni



Inferenze in OWL

Complessità

- Variazione sintattica di *SROIQ*
- *ALC* è una restrizione di *SROIQ*
- *ALC* è PSpace-hard

Reasoner

- Pellet
- Fact++
- HermiT
- ...

